in onion architecture/DDD, how to handle the rare case of one aggregate needing update another?

In Domain-Driven Design (DDD) and Onion Architecture, the core rule of aggregates is: **one transaction should only modify a single aggregate root.** When Aggregate A needs to trigger a change in Aggregate B, you have three primary ways to handle it, depending on your consistency requirements.

1. Domain Events & Eventual Consistency *(The Standard DDD Approach)*

If immediate consistency isn't an absolute, hard business requirement (which is true 95% of the time), use **Domain Events**.

How it works:

1. Aggregate A executes its business logic and raises a **Domain Event** (e.g., `OrderPlaced`).
2. The infrastructure/application layer commits Aggregate A's transaction and publishes the event.
3. An **Event Handler** listens for `OrderPlaced`, loads Aggregate B (`Inventory`), and calls Aggregate B's method to perform its update in a separate transaction.

Why it fits Onion Architecture: The domain layer defines the event, while the application/infrastructure layer handles the dispatching and side effects. Aggregate boundaries stay clean and decoupled.

2. Application Service Orchestration *(Immediate Consistency)*

If the business *strictly* demands that both updates succeed or fail together within the exact same database transaction, orchestrate it in the **Application Layer**.

How it works:

1. The **Application Service** (orchestrator) starts a transaction

1. The **Application Service** (orchestrator) starts a transaction.
2. It loads Aggregate A, invokes Aggregate A's business logic, and saves Aggregate A via its repository.
3. It loads Aggregate B, invokes Aggregate B's business logic, and saves Aggregate B via its repository.
4. The Application Service commits the transaction.

Important Note: Never let Aggregate A directly hold a reference to Aggregate B or call Aggregate B's repository inside its domain logic. Keep the dependency management in the Application Service layer.

3. Merge Aggregates *(Re-evaluating Boundaries)*

If Aggregate A and Aggregate B *must* always be updated together to maintain a strict business invariant, ask yourself if they are actually separate aggregates.

How it works:

Re-evaluate your domain boundaries. If they cannot exist in an eventually consistent state without breaking a core business rule, Aggregate B might actually be an entity *inside* Aggregate A's aggregate root.

Summary Checklist

Approach	Consistency	When to Use
Domain Events	Eventual	Default choice. Scales well and keeps aggregates decoupled.
App Service Orchestration	Immediate (Single Tx)	Strict business invariants requiring atomic updates across two aggregates.
Merge Aggregates	Immediate	You realize the two entities actually share a single boundary invariant.

can you give a few real world example of when one aggregate would need to update another?

Here are a few classic real-world scenarios where an action in one aggregate requires an update to another:

1. E-Commerce: Order Placement & Inventory Control

Aggregates involved: `Order` and `Inventory` (or `Product Stock`).

The Scenario: A customer places an order containing 2 units of a laptop.

The Interaction: * The `Order` aggregate processes the checkout and raises an `OrderPlaced` event.

The `Inventory` aggregate consumes this event and decrements the available stock count by 2.

Why keep them separate? High-volume systems can't afford to lock inventory rows inside the same database transaction as order creation. Allowing eventual consistency via domain events prevents stock lookup bottlenecks during high-traffic sales.

2. Banking: Wire Transfer Between Two Accounts

Aggregates involved: `BankAccount` (Source) and `BankAccount` (Destination).

The Scenario: Customer A transfers $500 to Customer B.

The Interaction: * `BankAccount A` validates that it has sufficient funds, deducts $500, and emits a `TransferInitiated` event (or completes its phase of an Application Service orchestration).

`BankAccount B` receives the incoming transfer and credits its balance by $500.

Why keep them separate? Each bank account is its own consistency boundary. Locking two separate accounts in a single long-lived database transaction across different users (or even different server instances/databases) creates huge concurrency issues.

3. Ride-Hailing (e.g., Uber): Trip Completion & Driver Status

Aggregates involved: `Trip` and `Driver`.

The Scenario: A driver completes a passenger's ride.

The Interaction:

The driver taps "End Trip". The `Trip` aggregate updates its status to `Completed`, calculates the fare, and emits a `TripCompleted` event.

The `Driver` aggregate listens for this event, updates the driver's current availability status to `Available`, and updates their daily earnings tally.

Why keep them separate? A driver's account state (earnings, availability, current location) and a specific passenger trip have completely different lifecycles and business rules.

4. SaaS & Identity: User Registration & Organization Quota

Aggregates involved: `User` and `Organization` (or `Account`).

The Scenario: A team admin invites a new member to their workspace on a tiered SaaS platform.

The Interaction:

The `User` aggregate is created in the system.

An event or application service updates the `Organization` aggregate to increment its `active_seats_used` counter. If the seat limit is reached, it flags the workspace as needing an upgrade.

Why keep them separate? An organization might have thousands of users. If every user creation locked the entire `Organization` aggregate, team members couldn't onboard simultaneously without hitting database locks.

could you make the argument that unless otherwise needed, always keep the update of multiple aggregates in a single transaction? It simplifies reasoning about the system and you don't need to handle rollbacks with eventually consistency?

Yes, you can absolutely make a strong, pragmatic argument for **single-transaction updates by default**, especially in mid-sized applications or non-distributed monoliths. Many software architects actually advocate for starting here until scale forces your hand.

Here is how that argument plays out:

1. Drastically Lower Cognitive Load

ACID transactions are a powerful abstraction. When you rely on a database transaction, the database handles the dirty work:

> **All-or-Nothing Guarantees:** You never have to worry about Aggregate A saving successfully while Aggregate B fails halfway through due to a network blip or constraint violation.
>
> **No Phantom States:** Users won't refresh their screen and see an order created but the inventory not yet reserved, avoiding customer confusion and complex UI state handling.

2. Eliminating Complex Compensating Logic (Sagas)

Eventual consistency sounds great in theory, but in practice, handling failure modes in asynchronous systems is notoriously hard:

> With eventual consistency, if Aggregate B fails to process an event, you must build **compensating transactions** (Sagas) to manually roll back Aggregate A.
>
> Writing, testing, and debugging retry queues, dead-letter queues, and compensating events adds massive overhead to the codebase—often far exceeding the effort of writing the actual business logic.

3. Simpler Infrastructure & Operations

Going all-in on event-driven eventual consistency usually requires dedicated infrastructure:

> Transactional Outbox patterns, message brokers (RabbitMQ, Kafka), and background event processors.
>
> Monitoring tools to track stuck messages or out-of-order execution.

Keeping updates within a single database transaction keeps your stack lean and your deployment pipeline simple.

4. Pragmatism Over Dogma (YAGNI)

The strict rule of *"One aggregate per transaction"* originated from high-scale distributed systems where databases literally couldn't handle cross-aggregate locks.

However, for 90% of business applications:

> Database load is modest.
>
> Relational databases (PostgreSQL, SQL Server) handle multi-table ACID transactions in milliseconds with minimal contention.
>
> Rearchitecting for eventual consistency before you actually hit database scaling bottlenecks is often a case of **Premature Optimization**.

The Takeaway

"Start with relational ACID safety, and only migrate to eventual consistency when performance or domain boundaries strictly require it."

The main trade-off to watch out for with this approach is **database lock contention**—if Aggregate A and Aggregate B are heavily modified by competing threads, single-transaction updates can cause performance bottlenecks or deadlocks. But until those bottlenecks manifest in profiling, prioritizing developer productivity and immediate consistency is a completely valid engineering choice.